# การเปรียบเทียบคุณภาพและประสิทธิภาพระหว่างแอพที่พัฒนาด้วยเอไอกับแอพที่พัฒนาโดยมนุษย์

Quality and Productivity Comparison: AI-Generated Apps vs Human-Developed Apps


**โดย**

B6601164 นายรชต เทียมโคกกรวด

อาจารย์ที่ปรึกษาโครงงาน

ผศ. ดร. ศุภกฤษฏิ์ นิวัฒนากูล


รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา

1101912 Project in Digital Technology II โครงงานเทคโนโลยีดิจิทัล

สาขาวิชาเทคโนโลยีดิจิทัล

สำนักวิชาศาสตร์และศิลป์ดิจิทัล มหาวิทยาลัยเทคโนโลยีสุรนารี

ประจำภาคการศึกษาที่ 2 ปีการศึกษา 2568

สารบัญ


เนื้อหา หน้าที่

บทที่ 1 บทนำ ..... 1

1. ชื่อโครงงาน ..... 1
2. ที่มาและความสำคัญ ..... 1
3. วัตถุประสงค์ ..... 2
4. ประโยชน์ที่คิดว่าจะได้รับ ..... 2

บทที่ 2 แนวคิด ทฤษฎีและงานวิจัยที่เกี่ยวข้อง ..... 3

1. ปัญญาประดิษฐ์เชิงสร้างสรรค์ (Generative AI) ..... 3
2. Large Language Models (LLMs) ..... 3
3. แอพพลิเคชั่นที่สร้างขึ้นด้วย AI (AI-Generative Application) : ..... 3
4. แอพพลิเคชั่นที่พัฒนาขึ้นด้วยมนุษย์ (Human-Develop Application) ..... 4
5. คุณภาพของซอฟต์แวร์ (Software Quality) ..... 4
6. ประสิทธิภาพในการพัฒนา (Productivity) ..... 4
7. การพัฒนาที่มี AI ในการช่วยเหลือ (AI-Assisted Development) ..... 4
8. Human-in-the-Loop (HITL) ..... 4
9. มาตรฐาน ISO/IEC 25010 ..... 5
10. เอกสารที่เกี่ยวข้อง ..... 5

บทที่ 3 เครื่องมือและวิธีการศึกษา ..... 7

1. รูปแบบการวิจัย ..... 7
2. ขอบเขตการศึกษา ..... 7
3. เครื่องมือที่ใช้ในการศึกษา ..... 7
4. ขั้นตอนการดำเนินการศึกษา ..... 8

บทที่ 4 ผลการดำเนินงาน ..... 9

1. กำหนดความต้องการ ..... 9

2. การพัฒนาแอพพลิเคชั่น ..... 11
3. ผลการทดลอง ..... 18
4. ผลด้านคุณภาพ ..... 23
บทที่ 5 สรุป อภิปรายผลการศึกษา และ บทคัดย่อ ..... 27
1. สรุปผลการศึกษา ..... 27
2. อภิปรายผลการศึกษา ..... 27
3. ข้อเสนอแนะ ..... 28
บทคัดย่อ ..... 29

# บทที่ 1
# บทนำ

## 1. ชื่อโครงงาน

การเปรียบเทียบคุณภาพและประสิทธิภาพระหว่างแอพที่พัฒนาด้วย AI กับแอพที่พัฒนาโดยมนุษย์
Quality and Productivity Comparison: AI-Generated Apps vs Human-Developed Apps

## 2. ที่มาและความสำคัญ

ปัจจุบันเทคโนโลยีปัญญาประดิษฐ์เชิงกำเนิด (Generative AI) โดยเฉพาะโมเดลภาษาขนาดใหญ่ (Large Language Models: LLMs) ได้เข้ามามีบทบาทสำคัญในการพัฒนาซอฟต์แวร์อย่างกว้างขวาง ทั้งการสร้างโค้ดอัตโนมัติ การช่วยออกแบบระบบ การทดสอบซอฟต์แวร์ และการเพิ่มประสิทธิภาพในกระบวนการพัฒนา ทำให้เกิดคำถามสำคัญว่าแอพพลิเคชันที่ถูกพัฒนาโดย AI สามารถทดแทนหรือแข่งขันกับแอพที่พัฒนาโดยมนุษย์ได้หรือไม่ ทั้งในด้านคุณภาพ (Quality) และประสิทธิภาพ (Productivity)

หลังจากการเปิดตัว GPT-3 ในปี 2020 และ GPT-4 ในปี 2023 อุตสาหกรรมซอฟต์แวร์มีการปรับตัวอย่างรวดเร็ว ธุรกิจจำนวนมากเริ่มใช้ AI เพื่อเร่งการผลิตซอฟต์แวร์ ลดต้นทุน และสร้างนวัตกรรมใหม่ ทำให้เกิดรูปแบบการพัฒนาที่หลากหลาย ตั้งแต่ AI ช่วยพัฒนา (AI-assisted development) ไปจนถึงแอพรุ่นใหม่ที่ถูกสร้างขึ้นโดย AI ทั้งหมด

อย่างไรก็ตาม แม้ว่า AI จะช่วยเพิ่มประสิทธิภาพในกระบวนการพัฒนาได้อย่างชัดเจน แต่ยังมีข้อถกเถียงเกี่ยวกับคุณภาพ ความถูกต้อง ความสามารถในการบำรุงรักษา (Maintainability) ความปลอดภัย (Security) และความคิดสร้างสรรค์เมื่อเทียบกับผลงานที่สร้างโดยมนุษย์ ดังนั้น การศึกษาที่มุ่งเปรียบเทียบคุณภาพและประสิทธิภาพระหว่างแอพที่สร้างด้วย AI และแอพที่พัฒนาโดยมนุษย์จึงมีความสำคัญต่อทั้งภาคการศึกษาและอุตสาหกรรม

ด้วยเหตุนี้ การวิจัยครั้งนี้จึงมุ่งเน้น การทดลองโดยใช้โจทย์พัฒนาแอพเดียวกัน แล้วเปรียบเทียบผลลัพธ์ระหว่างแอพที่สร้างด้วย AI และแอพที่พัฒนาโดยมนุษย์อย่างเป็นระบบ เพื่อให้ได้ข้อมูลเปรียบเทียบที่เป็นรูปธรรม ทั้งในด้านคุณภาพซอฟต์แวร์ตามมาตรฐาน ISO/IEC 25010 และด้านประสิทธิภาพการพัฒนา เช่น เวลา ข้อผิดพลาด ปริมาณการแก้ไขงาน และความต่อเนื่องของกระบวนการพัฒนา

## 3. วัตถุประสงค์

3.1 เพื่อเปรียบเทียบประสิทธิภาพ (Productivity) ของการพัฒนาแอพพลิเคชันระหว่างการใช้ AI เท่านั้นในการสร้างแอพและการพัฒนาโดยมนุษย์

3.2 เพื่อประเมินคุณภาพ (Quality) ของแอพที่สร้างด้วย AI และแอพที่พัฒนาโดยมนุษย์

## 4. ประโยชน์ที่คิดว่าจะได้รับ

4.1 เข้าใจและเห็นความแตกต่างด้านคุณภาพของแอพพลิเคชั่นที่พัฒนาโดยมนุษย์และ AI

4.2 แนวทางการนำวิธีการไปปรับใช้ในการพัฒนาเพื่อเพิ่มประสิทธิภาพ

4.3 ข้อมูลเชิงประจักษ์สำหรับการตัดสินใจนำ Generative AI เข้ามาใช้ในการพัฒนาซอฟต์แวร์

4.4 ยกระดับความรู้ทางวิชการด้านการพัฒนาซอฟต์แวร์ด้วย AI

# บทที่ 2
# แนวคิด ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การพัฒนาแอพพลิเคชั่นในยุคปัจจุบันกำลังเปลี่ยนแปลงอย่างรวดเร็ว จากรูปแบบการพัฒนาที่นักพัฒนานั้นต้องเขียนโค้ดด้วยตัวเอง ไปสู่การใช้เทคโนโลยีปัญญาประดิษฐ์สำหรับการช่วยออกแบบในหลายส่วน เพื่อให้ได้ทำความเข้าใจตรงกัน จึงขอนำเสนอแนวคิด ทฤษฎีและงานวิจัยที่เกี่ยวข้องดังต่อไปนี้

## 1. ปัญญาประดิษฐ์เชิงสร้างสรรค์ (Generative AI)

Generative AI หมายถึง เทคโนโลยีปัญญาประดิษฐ์ประเภทหนึ่งที่สามารถสร้างข้อมูล แนวคิด บทสนทนา รูปภาพและอื่นๆขึ้นมาได้ใหม่โดยอัตโนมัติ โดยอาศัยการเรียนรู้จากข้อมูลจำนวนมาก เช่น โมเดลของ GPT, Gemini, Claude เป็นต้น มีความสามารถในด้านการสร้างโค้ดและอกแบบซอฟต์แวร์ ทำให้เป็นส่วนนึงที่สำคัญสำหรับการพัฒนาซอฟต์แวร์ด้วย AI ทั้งในปัจจุบันและอนาคต

## 2. Large Language Models (LLMs)

Large Language Models (LLMs) เป็นหนึ่งในเทคโนโลยีหลักของ Generative AI โดยเป็นโมเดลภาษาขนาดใหญ่ที่ถูกฝึกด้วยข้อความจำนวนมหาศาลจากอินเทอร์เน็ต หนังสือ Source code และเอกสารต่าง ๆ ทำให้สามารถเข้าใจและสร้างข้อความหรือโค้ดที่มีความต่อเนื่องและมีตรรกะใกล้เคียงกับมนุษย์ได้ ตัวอย่างเช่น โมเดลที่ใช้ในเครื่องมืออย่าง ChatGPT, GitHub Copilot หรือเครื่องมือ AI coding assistant อื่น ๆ

- ในบริบทของการพัฒนาซอฟต์แวร์ LLM สามารถทำหน้าที่หลายอย่าง เช่น
- แปลงคำอธิบายความต้องการ (Requirements) ให้กลายเป็นโค้ดม็อคอัพหรือเดโม่
- ช่วยเขียน ฟังก์ชัน/คลาส ตามคำสั่งภาษาพูดทั่วไป
- แนะนำการแก้บั๊กหรือ Refactor โค้ด
- สร้างเทสเคสหรือโค้ดสำหรับการทดสอบอัตโนมัติ

## 3. แอพพลิเคชั่นที่สร้างขึ้นด้วย AI (AI-Generative Application)

แอพพลิเคชั่นที่สร้างขึ้นด้วย AI (AI-Generative Application) หมายถึงแอพพลิเคชั่นที่ถูกสร้างขึ้นทั้งหมดหรือบางส่วนด้วยปัญญาประดิษฐ์ เช่น การพัฒนาโค้ดทั้งไฟล์ การออกแบบ UI การสร้างฐานข้อมูล การเขียนเอกสาร การเขียนการทดสอบ

**4. แอพพลิเคชั่นที่พัฒนาขึ้นด้วยมนุษย์ (Human-Develop Application)**

แอพพลิเคชั่นที่พัฒนาขึ้นด้วยมนุษย์ (Human-Develop Application) หมายถึง แอพพลิเคชั่นที่นักพัฒนาหรือทีมพัฒนาได้สร้างขึ้นเองทั้งหมด โดยอาศัยความรู้ ความคิดสร้างสรรค์ การออกแบบเชิงระบบและประสบการณ์ตั้งแต่เริ่มต้นจนเสร็จสมบูรณ์ 2

**5. คุณภาพของซอฟต์แวร์ (Software Quality)**

คุณภาพของซอฟต์แวร์ (Software Quality) หมายถึง ชุดแนวคิดสำหรับใช้ในการประเมินความสมบูรณ์ของซอฟต์แวร์ โดยพิจารณาจากองค์ประกอบหลัก เช่น

5.1 Correctness ความถูกต้อง

5.2 Reliability ความน่าเชื่อถือ

5.3 Maintainability ความสามารถในการบำรุงรักษา

5.4 Security ความปลอดภัยในการใช้งาน

5.5 Efficiency มีประสิทธิภาพ

**6. ประสิทธิภาพในการพัฒนา (Productivity)**

ประสิทธิภาพในการพัฒนา (Productivity) หมายถึง ความสามารถในการพัฒนาซอฟต์แวร์ให้แล้วเสร็จอย่างรวดเร็ว มีต้นทุนต่ำ และใช้ทรัพยากรน้อย เช่น

6.1 เวลาที่ใช้ในการพัฒนา

6.2 จำนวนโค้ดที่สร้าง

6.3 จำนวนข้อผิดพลาด

6.4 ค่าใช้จ่ายในการพัฒนา

**7. การพัฒนาที่มี AI ในการช่วยเหลือ (AI-Assisted Development)**

AI-Assisted Development หมายถึง การพัฒนาซอฟต์แวร์ที่มนุษย์ยังมีการทำหน้าที่หลัก แต่มีAI คอยช่วยสนับสนุนบางส่วนเช่น แนะนำโค้ด ช่วยแก้บั๊ก เขียนเอกสาร ช่วยทดสอบ เป็นต้น ซึ่งเป็นการพัฒนาที่อยู่กึ่งกลางระหว่าง การพัฒนาด้วยมนุษย์ และ การใช้ AI

พัฒนาเต็มรูปแบบ

**8. Human-in-the-Loop (HITL)**

Human-in-the-Loop (HITL) หมายถึง รูปแบบที่ AI ทำงานจำนวนมาก เช่น สร้างโค้ดทั้งไฟล์หรือทั้งหน้าจอ แต่มนุษย์จะต้องเข้ามาตรวจสอบและรับรองผล ก่อนนำไปใช้งานจริง เป็นการวางมนุษย์ในบทบาทผู้ตรวจสอบคุณภาพ (Validator / Reviewer)

## 9. มาตรฐาน ISO/IEC 25010

ISO/IEC 25101 คือมาตรฐานสากลที่กำหนดแบบจำลองคุณภาพของระบบและซอฟต์แวร์ ซึ่งช่วยในการระบุ วัด และประเมินคุณภาพของซอฟต์แวร์ใน 8 คุณลักษณะดังนี้

1) Functional Suitability ความครบถ้วนและความถูกต้องของฟังก์ชัน
2) Performance Efficiency ความเร็ว การใช้ทรัพยากร
3) Compatibility ความสามารถในการทำงานร่วมกับระบบหรืออุปกรณ์อื่น
4) Usability ความง่ายต่อการใช้งาน
5) Reliability ความเสถียรและความถูกต้องในการทำงาน
6) Security ความปลอดภัยของข้อมูล
7) Maintainability ความง่ายในการแก้ไขและพัฒนาต่อ
8) Portability ความสามารถย้ายใช้งานข้ามระบบ

## 10. เอกสารที่เกี่ยวข้อง

**Future of Software Development with Generative AI (Sauvola et al., 2024)**

เป็นงานวิจัยที่อธิบายส่วนสำคัญไว้เรื่องการเปลี่ยนผ่านของอุตสาหกรรมซอฟต์แวร์ เมื่อเทคโนโลยี Generative AI และ Large Language Models (LLMs) ได้ถูกนำมาใช้ในกระบวนการพัฒนา นักวิจัยระบุว่าการเปิดตัวของ LLM ระดับสูงเช่น GPT-3 และ GPT-4 ได้เปลี่ยนแปลงกระบวนการทำงานของนักพัฒนาอย่างมีนัยยะสำคัญ ทั้งในแง่ของการพัฒนาโค้ด การออกแบบระบบ การทดสอบ ตลอดจนถึงการDebugและการแก้ไขปรับปรุงซอฟต์แวร์

งานวิจัยชี้ให้เห็นได้ว่า AI สามารถช่วยเพิ่มประสิทธิภาพการทำงานของนักพัฒนาได้ถึง 20–50% โดยเฉพาะงานที่ซ้ำซ้อน เช่น การเขียนโค้ดพื้นฐาน และการออกแบบหน้า UI นอกจากนี้งานวิจัยนี้มีความสำคัญอย่างยิ่งต่อการศึกษาในครั้งนี้ เนื่องจากชี้ให้เห็นถึงความแตกต่างระหว่าง “ความเร็วในการพัฒนา” และ “ความสามารถของ AI ในการสร้างโค้ด” ซึ่งเป็นรากฐานของการวิเคราะห์ประสิทธิภาพระหว่าง AI และมนุษย์ ขณะเดียวกันก็ชี้ให้เห็นข้อจำกัดด้านคุณภาพ เช่น ความถูกต้อง ความปลอดภัย และความจำเป็นของการตรวจสอบโดยมนุษย์ (Human-in-the-loop)

**A Comparative Study of Project Deliverable Efficiency and Quality in Thai Government Software Development Projects: ISO/IEC 29110 vs. 3-Phase Quality Inspection Method with ISO/IEC 29110**

งานวิจัยนี้มุ่งเน้นเปรียบเทียบคุณภาพและประสิทธิภาพของงานพัฒนาซอฟต์แวร์ในหน่วยงานภาครัฐ โดยนำมาตรฐาน ISO/IEC 29110 มาใช้เป็นกรอบสำหรับการตรวจสอบคุณภาพ ร่วมกับวิธีการตรวจสอบคุณภาพแบบ 3 ระยะ เพื่อประเมินความครบถ้วนถูกต้อง

ซึ่งจากงานวิจัยชี้ให้เห็นได้ว่า การใช้มาตรฐาน ISO/IEC 29110 ช่วยให้กระบวนการทำงานมีระบบระเบียบมากขึ้น คุณภาพของผลลัพธ์ดีขึ้นอย่างมีนัยยะสำคัญ งานวิจัยนี้มีความสำคัญต่อการศึกษาครั้งนี้ เพราะให้พื้นฐานด้านคุณภาพซอฟต์แวร์ และ ประสิทธิภาพกระบวนการ ซึ่งสามารถนำมาปรับใช้ในการประเมินคุณภาพงานของแอพที่สร้างขึ้นด้วย AI กับ งานที่มนุษย์พัฒนาเอง ได้อย่างเหมาะสมและตรงประเด็น

จากอ้างอิงบทความเหล่านี้แล้วนั้น ในงานวิจัยนี้ได้นำส่วนเรื่องมาตรฐานการประเมิน รูปแบบการประเมิน และได้นำความรู้ที่เกี่ยวกับเอไอ มาทำการทดลองในครั้งนี้ เพื่อเป็นการดูและสังเกตุว่าการใช้เอไอในการทำงานนั้นมีประสิทธิภาพและความถูกต้องมากแค่ไหน และเมื่อเปรียบเทียบกับความสามารถของมนุษย์แล้วนั้น เอไอมีประสิทธิภาพดีกว่าหรือไม่

# บทที่ 3
# เครื่องมือและวิธีการศึกษา

## 1. รูปแบบการวิจัย

การศึกษาครั้งนี้เป็นการวิจัยเชิงเปรียบเทียบ โดยมุ่งเน้นไปที่การวิเคราะห์ความแตกต่างระหว่าง แอพพลิคชั่นที่สร้างด้วยปัญญาประดิษฐ์เชิงกำหนด (AI-Generative Application) และ แอพพลิเคชั่นที่พัฒนาโดยมนุษย์ (Human-Develop Application) ซึ่งจะทำการเปรียบเทียบองค์ประกอบสำคัญสองอย่างคือ การเปรียบเทียบคุณภาพ (Quality) และ ประสิทธิภาพในการผลิต (Productivity)

## 2. ขอบเขตการศึกษา

การศึกษาจะทำการเขียนกำหนดความต้องการ และใช้ความต้องการชุดเดียวกันสำหรับในการพัฒนาทั้งสองแนวทาง เพื่อควบคุมตัวแปรและทำให้การเปรียบเทียบเป็นธรรมที่สุด โดยที่จะมีแอพพลิเคชั่น 2 ชุดในการพัฒนาครั้งนี้คือ ชุดที่ 1 แอพที่ใช้เอไอสร้างขึ้นมาทั้งหมด ชุดที่ 2 แอพที่มนุษย์พัฒนาด้วยตัวเองจากความรู้ด้านการเขียนโค้ดและออกแบบระบบ

## 3. เครื่องมือที่ใช้ในการศึกษา

### 3.1 เครื่องมือสำหรับการสร้างซอฟต์แวร์

1) Gemini 3 Pro สำหรับการสร้างโค้ด ออกแบบหน้าตา UI และวิเคราะห์ข้อผิดพลาดสำหรับการออกแบบในการใช้พัฒนา AI-Generative Application
2) Visual Studio Code สำหรับการพัฒนา เขียนโค้ด
3) Framework หรือ Library สำหรับพัฒนาตาม Requirement

### 3.2 เครื่องมือสำหรับประเมินคุณภาพซอฟต์แวร์

ใช้ มาตรฐาน ISO/IEC 25010 เป็นกรอบหลักในการประเมินคุณภาพ โดยวัดตาม 8 หมวดดังนี้

1) Functional Suitability ความครบถ้วนและความถูกต้องของฟังก์ชัน
2) Performance Efficiency ความเร็ว การใช้ทรัพยากร
3) Compatibility ความสามารถในการทำงานร่วมกับระบบหรืออุปกรณ์อื่น
4) Usability ความง่ายต่อการใช้งาน
5) Reliability ความเสถียรและความถูกต้องในการทำงาน
6) Security ความปลอดภัยของข้อมูล
7) Maintainability ความง่ายในการแก้ไขและพัฒนาต่อ
8) Portability ความสามารถย้ายใช้งานข้ามระบบ

สามารถนำมาใช้กับโครงงานจริงดังนี้

- Functional test ใช้สำหรับวัด Functional Suitability
- Load test ใช้สำหรับวัด Performance Efficiency
- Static analysis ใช้สำหรับวัด Maintainability / Security
- Usability checklist ใช้สำหรับวัด Usability
- ในส่วนของ Portability อาจจะยังไม่ได้นำมาใช้เนื่องจาก การทดสอบและพัฒนานั้นจะทำการพัฒนาบางฟังชั่น

**3.3 เครื่องมือประเมินประสิทธิภาพในการผลิต**

1) เวลาที่ใช้ในการพัฒนา
2) จำนวนโค้ดที่สร้าง
3) จำนวนข้อผิดพลาดที่พบ
4) ค่าใช้จ่ายหรือจำนวนรอบที่แก้ไข

**4. ขั้นตอนการดำเนินการศึกษา**

**ขั้นตอนที่ 1 กำหนดความต้องการ**

สร้าง Requirement Document ตามรูปแบบเดียวกัน สำหรับการพัฒนาด้วยเอไอ

(AI-Generative Application) และ แอพที่พัฒนาโดยมนุษย์ (Human-Develop Application)

**ขั้นตอนที่ 2 การพัฒนาแอพพลิเคชั่น**

ใช้เอไอสร้างโค้ด ฐานข้อมูล ส่วนประกอบ หน้าตาแอพพลิเคชั่น ทั้งหมดทุกส่วนเพื่อทดสอบความสามารถของเอไอ สำหรับการเขียนโค้ดอัตโนมัติ แล้วทำการบันทึกผลทุกขั้นตอน

เริ่มดำเนินการพัฒนาแอพพลิเคชั่นด้วยตัวข้าพเจ้าที่เป็นนักศึกษา เขียนโค้ดทั้งหมดตาม Requirement ใช้ความรู้และประสบการณ์ในการออกแบบ พัฒนาแอพขึ้นมาทั้งหมดแล้วทำการบันทึกผลทุกขั้นตอน

**ขั้นตอนที่ 3 การทดสอบและประเมิน**

ทำการทดสอบตามมาตรฐาน ISO/IEC 25010 เพื่อประเมินสำหรับข้อมูลของการวิจัย

ทำการบันทึกเวลา จำนวนโค้ด บั๊ก และจำนวนรอบแก้ไขตามจริงเพื่อประเมินสำหรับข้อมูลของการวิจัย

# บทที่ 4

## ผลการดำเนินงาน

### 1. กำหนดความต้องการ

ส่วนการกำหนดความต้องการขึ้นมานั้น จะทำการกำหนดความต้องการที่เป็นมาตรฐานของแอพทั้งหมดสำหรับใช้ในการพัฒนาแอพด้วยเอไอและมนุษย์ เพื่อให้แนวทางของการทำแอพออกมาตรงกัน ได้ทำการเขียนออกมาดังนี้

System Requirements

**ความต้องการด้านฟังก์ชัน (Functional Requirements)**

1.1. ระบบคัดกรองเมนูอาหาร

1.1.1.ผู้ใช้สามารถเลือกหมวดอาหารที่ต้องการได้

1.1.2.ระบบสามารถเลือกหลายหมวดพร้อมกันได้

1.1.3.ระบบต้องมีการตรวจสอบว่าผู้ใช้เลือกหมวดหรือไม่

1.2. ระบบเลือกโหมดการทำงาน

1.2.1.ผู้ใช้สามารถเลือกโหมดได้ว่าจะใช้โหมดสุ่มหรือตัดสินใจ

1.3. ระบบการทำงานโหมดสุ่ม

1.3.1.ระบบทำการสุ่มเมนู 1 รายการ

1.3.2.แสดงผลลัพธ์พร้อมข้อมูล

1.3.3.ผู้ใช้กดสุ่มใหม่ได้จนกว่าจะพอใจ

1.4. ระบบการทำงานโหมดเลือกตัดสินใจ

1.4.1.ระบบดึงรายการมาจับคู่กันทีละคู่

1.4.2.แสดงผลหน้าจอเป็นสองฝั่งให้เลือก

1.4.3.ผู้ใช้เลือกเมนูฝั่งไหน ให้ฝั่งที่โดนเลือกเข้าไปรอบต่อไป

1.4.4.ระบบวนลูปจับคู่ผู้ชนะรอบก่อนหน้าไปเรื่อยๆ จนถึงเมนูสุดท้าย

1.5. การจบการทำงาน

1.5.1.ได้เมนูสุดท้ายแล้ว ต้องกดยืนยันและกดปุ่ม”กลับไปหน้าหลัก”

**ความต้องการที่ไม่ใช่ฟังชั่น (Non-Functional Requirement)**

2.1ด้านการใช้งาน

2.1.1รองรับการใช้งานบนโทรศัพท์

2.1.2แสดงผลรวดเร็วชัดเจน

2.2ด้านประสิทธิภาพ

2.2.1พัฒนาด้วย React Native

2.2.2การประมวลผลจับคู่ที่รวดเร็ว

2.3ด้านข้อมูล

2.3.1Hardcoded Data ใช้การเขียนโค้ดสำหรับเก็บข้อมูลอาหารเพื่อเลี่ยงการใช้งาน Database และเพื่อให้ผลการดำเนินงานออกมาตรงกันมากที่สุด ได้ทำการกำหนดไลบรารี่เสริมเพิ่มเติมดังนี้

react-native-paper

expo-router

@expo-google-fonts/kanit

expo-font

@react-native-async-storage/async-storage

expo-file-system

expo-sharing

react-native-reanimated

ทั้งนี้ทั้งนั้น ไลบรารี่เหล่านี้สามารถติดตั้งเพิ่ม หรือ ลบบางอันที่ไม่ได้ใช้งานจริงๆได้

**2. การพัฒนาแอพพลิเคชั่น**

ในระหว่างการพัฒนานั้น จะแบ่งออกเป็นสองโฟลเดอร์คือ kin-rai-dee-ai คือ แอพที่พัฒนาโดย Ai และ kin-rai-dee คือแอพที่พัฒนาโดยมนุษย์

**การพัฒนาด้วย Ai**

การพัฒนาด้วยเอไอนั้น จะทำการสั่งการเป็น Prompt ทั้งหมดและมีPrompt ที่ใช้ดังนี้

**Prompt 1.**

| |
|---|
| **แปลง Requirement เหล่านี้เป็นPrompt สำหรับเตรียมทำเว็บไซต์** |
| System Requirements |
| **1. ความต้องการด้านฟังก์ชัน (Functional Requirements)** |
| **1.1. ระบบคัดกรองเมนูอาหาร** |
| **1.1.1.ผู้ใช้สามารถเลือกหมวดอาหารที่ต้องการได้** |
| **1.1.2.ระบบสามารถเลือกหลายหมวดพร้อมกันได้** |
| **1.1.3.ระบบต้องมีการตรวจสอบว่าผู้ใช้เลือกหมวดหรือไม่** |
| **1.2. ระบบเลือกโหมดการทำงาน** |
| **1.2.1.ผู้ใช้สามารถเลือกโหมดได้ว่าจะใช้โหมดสุ่มหรือตัดสินใจ** |
| **1.3. ระบบการทำงานโหมดสุ่ม** |
| **1.3.1.ระบบทำการสุ่มเมนู1 รายการ** |
| **1.3.2.แสดงผลลัพธ์พร้อมข้อมูล** |
| **1.3.3.ผู้ใช้กดสุ่มใหม่ได้จนกว่าจะพอใจหรือตามจำนวนที่กำหนดไว้** |
| **1.4. ระบบการทำงานโหมดเลือกตัดสินใจ** |
| **1.4.1.ระบบดึงรายการมาจับคู่กันทีละคู่** |
| **1.4.2.แสดงผลหน้าจอเป็นสองฝั่งให้เลือก** |
| **1.4.3.ผู้ใช้เลือกเมนูฝั่งไหน** |
| **ให้ฝั่งที่โดนเลือกเข้าไปรอบต่อไป** |
| **1.4.4.ระบบวนลูปจับคู่ผู้ชนะรอบก่อนหน้าไปเรื่อยๆ** |
| **จนถึงเมนูสุดท้าย** |
| **1.5. การจบการทำงาน** |
| **1.5.1.ได้เมนูสุดท้ายแล้ว** |
| **ต้องกดยืนยันและกดปุ่ม”กลับไปหน้าหลัก”** |
| **2. ความต้องการที่ไม่ใช่ฟังชั่น (Non-Functional Requirement)** |
| **2.1. ด้านการใช้งาน** |

2.1.1.รองรับการใช้งานบนโทรศัพท์

2.1.2.แสดงผลรวดเร็วชัดเจน

2.2. ด้านประสิทธิภาพ

2.2.1.พัฒนาด้วย React Native

2.2.2.การประมวลผลจับคู่

2.3. ด้านข้อมูล

2.3.1.Hardcoded Data ใช้การเขียนโค้ดสำหรับเก็บข้อมูลอาหารเพื่อเลี่ยงการใช้งาน Database

Prompt 2.

ใช้ TechStack

React Native และใช้Library ดังนี้

react-native-paper

expo-router

expo-sensors

@expo-google-fonts/kanit

expo-font

@react-native-async-storage/async-storage

expo-file-system

expo-sharing

react-native-reanimated

Prompt 3.

Act as a Senior React Native Developer. I need you to build a **Food Decision App** using **Expo**.

**Tech Stack & Libraries:**

- **Core:** React Native (Expo SDK 50+), Typescript (optional but preferred).
- **Navigation:** `expo-router` (File-based routing).
- **UI Component:** `react-native-paper` (Material Design).

- **Typography:** `@expo-google-fonts/kanit` & `expo-font` (Apply "Kanit" font globally).
- **Storage:** `@react-native-async-storage/async-storage` (To save user preference/history).
- **Sensors:** `expo-sensors` (Implement "Shake" gesture to re-roll in Random Mode).
- **File & Sharing:** `expo-file-system`, `expo-sharing` (To share the final result image).
- **Animations:** `react-native-reanimated`.
- **Data:** Hardcoded JSON (No external API).

**System Requirements & User Flow:**

**1. Data & Theme:**

- Create a `data.js` file with a hardcoded array of food items (id, name, category, image_url).
- Configure `react-native-paper` theme to use the **Kanit** font family for all text.

**2. Screen 1: Home & Filter (Route: `/`)**

- **UI:** Display categories as `Chip` or Checkboxes (Multi-select allowed).
- **Logic:** User selects desired categories. Validate that at least one is selected.
- **Action:** Button to proceed to Mode Selection.

**3. Screen 2: Mode Selection (Route: `/mode-select`)**

- **UI:** Two large cards/buttons:
  1. **Random Mode** (Single spin).
  2. **Tournament Mode** (Versus battle).

**4. Screen 3: Random Mode (Route: `/random`)**

- **Logic:** Randomly pick 1 item from the filtered list.
- **Feature:** Use `expo-sensors` (Accelerometer). If the user physically shakes the phone, trigger a re-roll.
- **UI:** Display the food with a smooth animation (`react-native-reanimated`). Include a manual "Random Again" button.
- **Action:** "Select This" button proceeds to Result.

**5. Screen 4: Tournament Mode (Route: `/tournament`)**

- **Logic:** Pair items 1vs1. Winner stays, Loser goes. Loop until 1 remains.

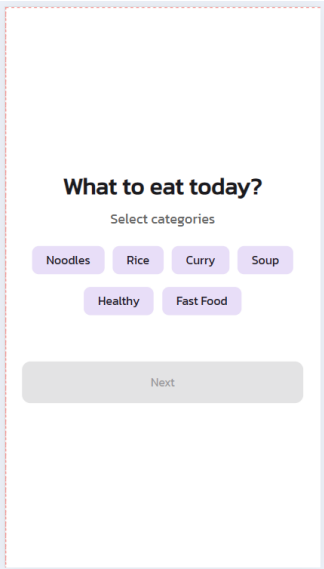
- **UI:** Split screen (Top vs Bottom).
- **Interaction:** User taps the preferred food. Use `react-native-reanimated` for transition effects when a choice is made.

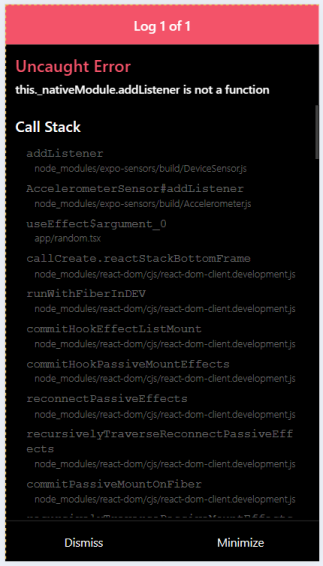
**6. Screen 5: Result (Route: `/result`)**

- **UI:** Show the winning food item clearly.
- **Feature:** Add a **"Share"** button. When clicked, capture the view (or generate a message) and use `expo-sharing` to share it to social media.
- **Action:** "Back to Home" button (resets the stack).

**Deliverables:**

- Provide the folder structure for `expo-router` (e.g., `app/_layout.tsx`, `app/index.tsx`, etc.).
- Provide the code for the main screens.
- Ensure the code handles the `Kanit` font loading correctly (using `useFonts`).
- Ensure styling follows `react-native-paper` guidelines.

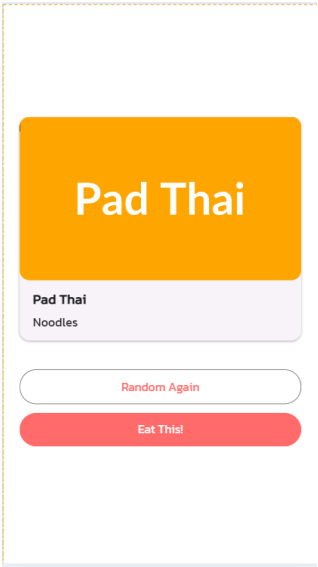


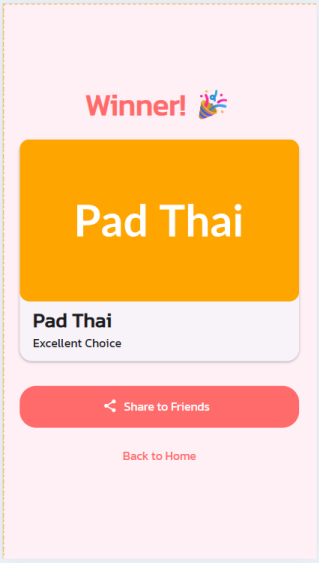



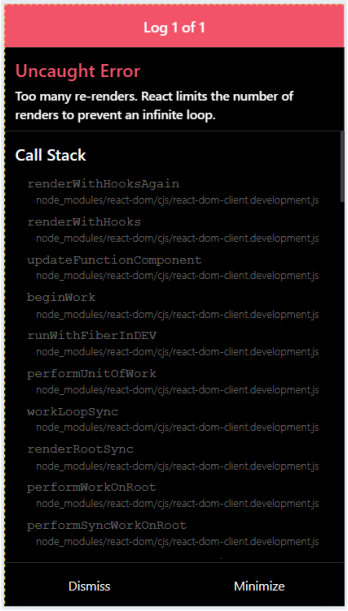

ได้ผลแอพออกมาดังนี้แต่มีเออเร่อในการใช้งาน

Prompt 4.

| |
|---|
| TypeError: this._nativeModule.addListener is not a function |




แอพใช้งานได้แล้ว แต่เมื่อกดใช้งานไปเรื่อยๆ มีปัญหาดังนี้

Prompt 5.

| |
|---|
| Error: Too many re-renders. React limits the number of renders to prevent an infinite loop. |

ใช้งานได้ปกติแล้ว แต่มีการกดได้แค่1-2ครั้งแล้วผลขึ้น

Prompt 6.

| |
|---|
| **กดได้1ถึง2ครั้ง แอพก็ทำการประกาศผลชนะทันที ยังไม่ได้มีการเลือกอันอื่นต่อเลย เกิดจากอะไร** |

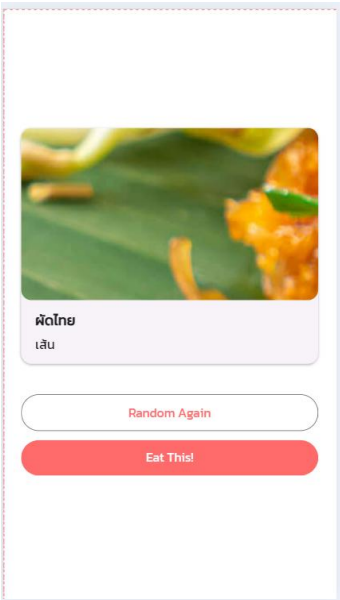
ใช้งานได้ปกติทุกอย่าง

Prompt 7.

| |
|---|
| **เพิ่มเมนูเป็นหมวดละ 5 เมนู ที่หากินได้ง่ายในชีวิตประจำวัน** |

Prompt 8.

| |
|---|
| **ต้องการใส่รูปเหล่านี้ลงไปในแต่ละเมนู** |




รูปขึ้นแล้วแต่มีปัญหาคือขนาดของรูปแสดงออกมาผิดพลาด

Prompt 9

**แก้ไขการจัดรูป เนื่องจากบางเมนูมีรูปที่ใหญ่เกินไป ใช้วิธีการย่อรูปให้เหลือเท่าขนาดกรอบ**

แอพสามารถใช้งานได้ปกติแล้ว มีรูปขึ้น กดใช้ฟังชั่นต่างๆได้ แต่เนื่องจากทำ Cross-Platform ด้วย เมื่อลองใช้งานดูบนมือถือเกิดปัญหาดังนี้

Prompt 10

**แก้ไขอย่างไร**

เอไอแนะนำวิธีการแก้ปัญหา ถือว่าแอพใช้งานได้เสร็จสมบูรณ์

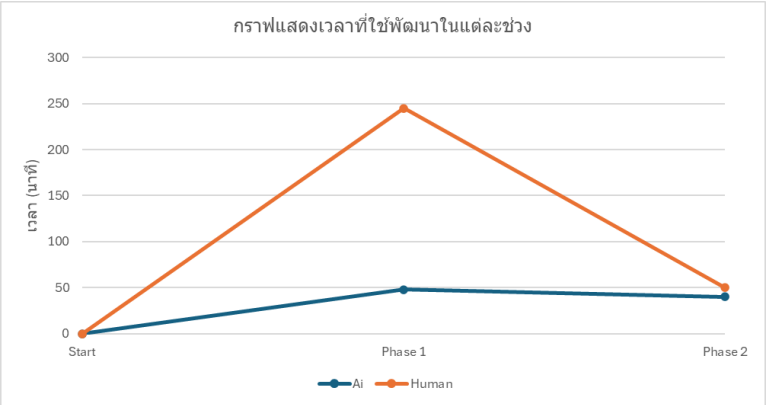
## 3. ผลการทดลอง

### 3.1 เวลาที่ใช้ในการพัฒนา

การพัฒนาแอพพลิเคชั่นได้แบ่งการพัฒนาออกเป็นสองระยะคือ

Phase 1 ระยะเวลาที่ใช้สำหรับการพัฒนาโปรแกรม

Phase 2 ระยะเวลาที่ใช้ในการตกแต่งโปรแกรมและแก้ปัญหา




| Ai-Development | | | |
|---|---|---|---|
| No. | Functional Requirement | Phase 1 | Phase 2 |
| 1 | ผู้ใช้สามารถเลือกหมวดอาหารที่ต้องการได้ | 48 | 40 |
| 2 | ระบบสามารถเลือกหลายหมวดพร้อมกันได้ | | |
| 3 | ระบบต้องมีการตรวจสอบว่าผู้ใช้เลือกหมวดหรือไม่ | | |
| 4 | ผู้ใช้สามารถเลือกโหมดได้ว่าจะใช้โหมดสุ่มหรือตัดสินใจ | | |
| 5 | ระบบทำการสุ่มเมนู 1 รายการ | | |
| 6 | แสดงผลลัพธ์พร้อมข้อมูล | | |
| 7 | ผู้ใช้กดสุ่มใหม่ได้จนกว่าจะพอใจ | | |
| 8 | ระบบดึงรายการมาจับคู่กันทีละคู่ | | |
| 9 | แสดงผลหน้าจอเป็นสองฝั่งให้เลือก | | |
| 10 | ผู้ใช้เลือกเมนูฝั่งไหน ให้ฝั่งที่โดนเลือกเข้าไปรอบต่อไป | | |
| 11 | ระบบวนลูปจับคู่ผู้ชนะรอบก่อนหน้าไปเรื่อยๆ จนถึงเมนูสุดท้าย | | |
| 12 | ได้เมนูสุดท้ายแล้ว ต้องกดยืนยันและกดปุ่ม"กลับไปหน้าหลัก" | | |
| | Non-Functional Requirement | | |
| 1 | รองรับการใช้งานบนโทรศัพท์ | | |
| 2 | แสดงผลรวดเร็วชัดเจน | | |
| 3 | พัฒนาด้วย React Native | | |
| 4 | การประมวลผลจับคู่ที่รวดเร็ว | | |
| 5 | Hardcoded Data โค้ดสำหรับเก็บข้อมูล | | |
| Human | | | |
| No. | Functional Requirement | Phase 1 | Phase 2 |
| 1 | ผู้ใช้สามารถเลือกหมวดอาหารที่ต้องการได้ | 245 | 50 |
| 2 | ระบบสามารถเลือกหลายหมวดพร้อมกันได้ | | |
| 3 | ระบบต้องมีการตรวจสอบว่าผู้ใช้เลือกหมวดหรือไม่ | | |
| 4 | ผู้ใช้สามารถเลือกโหมดได้ว่าจะใช้โหมดสุ่มหรือตัดสินใจ | | |
| 5 | ระบบทำการสุ่มเมนู 1 รายการ | | |
| 6 | แสดงผลลัพธ์พร้อมข้อมูล | | |
| 7 | ผู้ใช้กดสุ่มใหม่ได้จนกว่าจะพอใจ | | |
| 8 | ระบบดึงรายการมาจับคู่กันทีละคู่ | | |
| 9 | แสดงผลหน้าจอเป็นสองฝั่งให้เลือก | | |
| 10 | ผู้ใช้เลือกเมนูฝั่งไหน ให้ฝั่งที่โดนเลือกเข้าไปรอบต่อไป | | |
| 11 | ระบบวนลูปจับคู่ผู้ชนะรอบก่อนหน้าไปเรื่อยๆ จนถึงเมนูสุดท้าย | | |
| 12 | ได้เมนูสุดท้ายแล้ว ต้องกดยืนยันและกดปุ่ม"กลับไปหน้าหลัก" | | |
| | Non-Functional Requirement | | |
| 1 | รองรับการใช้งานบนโทรศัพท์ | | |
| 2 | แสดงผลรวดเร็วชัดเจน | | |
| 3 | พัฒนาด้วย React Native | | |
| 4 | การประมวลผลจับคู่ที่รวดเร็ว | | |
| 5 | Hardcoded Data โค้ดสำหรับเก็บข้อมูล | | |

## 3.2 ผลการทดสอบการทำงานฟังชั่น

ทำการทดสอบโดยทำการสร้างเคสสำหรับทดสอบ เพื่อทดสอบฟังชั่นการทำงานมา 7 เคส และนำเคสสำหรับการทดสอบนี้ไปใช้สำหรับทดสอบทั้ง 2 โปรแกรมที่พัฒนามาใช้ทดสอบบนมาตรฐานเดียวกัน ได้ดังนี้

ผลการทดสอบแอพที่ใช้ Ai พัฒนา

| Ai-Develop | | | | | |
|---|---|---|---|---|---|
| **Test ID** | **ชื่อการทดสอบ** | **ขั้นตอนการทดสอบ (Steps)** | **ผลลัพธ์ที่คาดหวัง (Expected Result)** | **ผลลัพธ์จริง** | **สถานะ** |
| **TC-001** | โหมดสุ่มอาหาร (Random Mode) | 1. กดปุ่ม "สุ่มเลย" จากหน้าหลัก<br>2. รอระบบประมวลผล | ระบบแสดงรูปภาพและชื่อเมนูอาหาร 1 เมนูขึ้นมา | แสดงผลถูกต้อง | ผ่าน |
| **TC-002** | โหมดโหวตเลือก (Decision Mode) | 1. กดปุ่ม "โหมดตัดสินใจ"<br>2. ระบบแสดงรูปอาหาร 2 รูปซ้ายขวา | ต้องมีรูปภาพขึ้นทั้ง 2 ฝั่ง และชื่อเมนูถูกต้อง | แสดงผลถูกต้อง | ผ่าน |
| **TC-003** | การเลือกผู้ชนะในแต่ละรอบ | ในหน้า Decision กดเลือกรูปฝั่ง "ซ้าย" | 1. รูปฝั่งซ้ายหายไป<br>2. ระบบเปลี่ยนคู่เทียบใหม่ขึ้นมาทันที | เปลี่ยนคู่ถูกต้อง | ผ่าน |
| **TC-004** | กรณีเหลือตัวเลือกเดียว (Bug Fix) | ทดสอบเลือกไปเรื่อยๆ<br>จนเหลือผู้ชนะ 1 คนสุดท้าย | ระบบต้อง ไม่ Error และตัดเข้าหน้า ResultsPage ทันที | ตัดเข้าหน้าผลลัพธ์ได้ถูกต้อง | ผ่าน |
| **TC-005** | หน้าแสดงผลลัพธ์ (Result Page) | ตรวจสอบข้อมูลในหน้าสุดท้าย | แสดงรูปเมนูที่ชนะ | แสดงถูกต้อง | ผ่าน |
| **TC-006** | ปุ่มแชร์ (Share) | กดปุ่ม "แชร์เมนู" | ระบบเปิดหน้าต่างแชร์รูปภาพของ | เปิดหน้าต่างแชร์ได้ | ผ่าน |
| **TC-007** | ปุ่มกลับหน้าหลัก (Home) | กดปุ่ม "กลับหน้าแรก" | ระบบเคลียร์ค่าผู้ชนะ และกลับไปหน้า Home | กลับหน้าแรกได้ | ผ่าน |

ผลการทดสอบแอพที่ใช้มนุษย์พัฒนา

| Human-Develop | | | | | |
|---|---|---|---|---|---|
| **Test ID** | **ชื่อการทดสอบ** | **ขั้นตอนการทดสอบ (Steps)** | **ผลลัพธ์ที่คาดหวัง (Expected Result)** | **ผลลัพธ์จริง** | **สถานะ** |
| **TC-001** | โหมดสุ่มอาหาร (Random Mode) | 1. กดปุ่ม "สุ่มเลย" จากหน้าหลัก<br>2. รอระบบประมวลผล | ระบบแสดงรูปภาพและชื่อเมนูอาหาร 1 เมนูขึ้นมา | แสดงผลถูกต้อง | ผ่าน |
| **TC-002** | โหมดโหวตเลือก (Decision Mode) | 1. กดปุ่ม "โหมดตัดสินใจ"<br>2. ระบบแสดงรูปอาหาร 2 รูปซ้ายขวา | ต้องมีรูปภาพขึ้นทั้ง 2 ฝั่ง และชื่อเมนูถูกต้อง | แสดงผลถูกต้อง | ผ่าน |
| **TC-003** | การเลือกผู้ชนะในแต่ละรอบ | ในหน้า Decision กดเลือกรูปฝั่ง "ซ้าย" | 1. รูปฝั่งซ้ายหายไป<br>2. ระบบเปลี่ยนคู่เทียบใหม่ขึ้นมาทันที | เปลี่ยนคู่ถูกต้อง | ผ่าน |
| **TC-004** | กรณีเหลือตัวเลือกเดียว (Bug Fix) | ทดสอบเลือกไปเรื่อยๆ<br>จนเหลือผู้ชนะ 1 คนสุดท้าย | ระบบต้อง ไม่ Error และตัดเข้าหน้า ResultsPage ทันที | ตัดเข้าหน้าผลลัพธ์ได้ถูกต้อง | ผ่าน |
| **TC-005** | หน้าแสดงผลลัพธ์ (Result Page) | ตรวจสอบข้อมูลในหน้าสุดท้าย | แสดงรูปเมนูที่ชนะ พร้อมข้อความแสดงความยินดี | แสดงถูกต้อง | ผ่าน |
| **TC-006** | ปุ่มแชร์ (Share) | กดปุ่ม "แชร์เมนูอร่อยๆ" | ระบบเปิดหน้าต่างแชร์รูปภาพของ OS (Android/iOS) | เปิดหน้าต่างแชร์ได้ | ผ่าน |
| **TC-007** | ปุ่มกลับหน้าหลัก (Home) | กดปุ่ม "กลับหน้าแรก" | ระบบเคลียร์ค่าผู้ชนะ และกลับไปหน้า Home | กลับหน้าแรกได้ | ผ่าน |

### 3.3 ผลการทดสอบด้านการใช้งาน และ ความปลอดภัย

ทำการทดสอบโดยทำการสร้างเคสสำหรับทดสอบ สำหรับทดสอบเรื่องความปลอดภัยและด้านการใช้งานมา เพื่อให้ได้เคสสำหรับทดสอบที่เป็นมาตรฐานตรงกัน โดยจะประเมินด้วยผ่านและไม่ผ่าน ได้ผลออกมาดังนี้

ผลการทดสอบแอพที่ใช้ Ai พัฒนา

| Ai-Develop | | | | |
|---|---|---|---|---|
| **หมวดหมู่** | **รายการตรวจสอบ (Checklist)** | **เกณฑ์ที่กำหนด** | **ผลการตรวจสอบ** | **หมายเหตุ** |
| **Usability** | ขนาดของปุ่มกด (Touch Target) | ปุ่มมีความสูงอย่างน้อย 44px (ตามมาตรฐาน Mobile UI) | ผ่าน | ปุ่มกดง่าย ไม่เล็กเกินไป |
| **Usability** | ขนาดตัวอักษร (Readability) | ตัวหนังสือชื่ออาหารต้องอ่านง่าย ไม่ต้องเพ่ง | ผ่าน | ใช้ Font Size 18-24px |
| **Usability** | ความเข้าใจในการใช้งาน (Learnability) | ผู้ใช้รู้ว่าต้องกดรูปภาพเพื่อเลือก | ผ่าน | UI ออกแบบให้เหมือนการโหวต |
| **Usability** | การแจ้งเตือน (Feedback) | มี Loading หรือ Transition เมื่อเปลี่ยนหน้า | ผ่าน | ใช้ React Native Reanimated |
| **Security** | การขอสิทธิ์เข้าถึง (Permissions) | ไม่มีการขอ Location หรือ Contact โดยไม่จำเป็น | ผ่าน | ขอแค่ Storage เพื่อแชร์รูป |
| **Security** | ข้อมูลความลับ (Sensitive Data) | ไม่มี API Key หรือรหัสผ่านฝังใน Code | ผ่าน | ข้อมูลเป็น Local Data (Offline) |

ผลการทดสอบแอพที่ใช้มนุษย์พัฒนา

| Human-Develop | | | | |
|---|---|---|---|---|
| **หมวดหมู่** | **รายการตรวจสอบ (Checklist)** | **เกณฑ์ที่กำหนด** | **ผลการตรวจสอบ** | **หมายเหตุ** |
| **Usability** | ขนาดของปุ่มกด (Touch Target) | ปุ่มมีความสูงอย่างน้อย 44px (ตามมาตรฐาน Mobile UI) | ผ่าน | ปุ่มกดง่าย ไม่เล็กเกินไป |
| **Usability** | ขนาดตัวอักษร (Readability) | ตัวหนังสือชื่ออาหารต้องอ่านง่าย ไม่ต้องเพ่ง | ผ่าน | ใช้ Font Size 18-24px |
| **Usability** | ความเข้าใจในการใช้งาน (Learnability) | ผู้ใช้รู้ว่าต้องกดรูปภาพเพื่อเลือก | ผ่าน | UI ออกแบบให้เหมือนการโหวต |
| **Usability** | การแจ้งเตือน (Feedback) | มี Loading หรือ Transition เมื่อเปลี่ยนหน้า | ผ่าน | ใช้ React Native Reanimated |
| **Security** | การขอสิทธิ์เข้าถึง (Permissions) | ไม่มีการขอ Location หรือ Contact โดยไม่จำเป็น | ผ่าน | ขอแค่ Storage เพื่อแชร์รูป |
| **Security** | ข้อมูลความลับ (Sensitive Data) | ไม่มี API Key หรือรหัสผ่านฝังใน Code | ผ่าน | ข้อมูลเป็น Local Data (Offline) |

## 3.4 ผลการทดสอบความเสถียรและประสิทธิภาพ

ทำการทดสอบโดยทำการสร้างเคสสำหรับทดสอบ สำหรับทดสอบเรื่องความเสถียรและประสิทธิภาพมา เพื่อให้ได้เคสสำหรับทดสอบที่เป็นมาตรฐานตรงกัน โดยจะประเมินด้วยผ่านและไม่ผ่าน ได้ผลออกมาดังนี้

ผลการทดสอบแอพที่ใช้ Ai พัฒนา

| Ai-Develop | | | | | |
|---|---|---|---|---|---|
| **Test ID** | **หัวข้อการทดสอบ** | **วิธีการทดสอบ** | **เกณฑ์ที่ผ่าน (Criteria)** | **ค่าที่วัดได้ / ผลลัพธ์** | **สรุป** |
| **PF-01** | Memory Usage (RAM) | เปิดหน้า Performance Monitor และใช้งานแอป 5 นาที | RAM ต้องไม่เกิน 300MB และไม่เพิ่มขึ้นเรื่อยๆ | เฉลี่ย 200-220 MB | Pass |
| **PF-02** | Frame Rate (FPS) | เลื่อนหน้าจอไปมา หรือกดเปลี่ยนรูปภาพ | UI FPS ต้องเลี้ยงอยู่ที่ 55-60 FPS | 59-60 FPS | Pass |
| **RL-01** | Stress Test (Monkey Test) | กดปุ่มตัวเลือก "ซ้าย/ขวา" สลับกันรัวๆ 20 ครั้ง | แอปต้องไม่เด้งหลุด (No Crash) | แอปทำงานต่อได้ปกติ | Pass |
| **RL-02** | Background Test | ยุบหน้าจอไปเล่นแอปอื่น แล้วกลับเข้ามาใหม่ | แอปต้องแสดงผลหน้าเดิม ไม่รีเซ็ตหรือจอดำ | กลับมาหน้าเดิมได้ | Pass |

ผลการทดสอบแอพที่ใช้มนุษย์พัฒนา

| Human-Develop | | | | | |
|---|---|---|---|---|---|
| **Test ID** | **หัวข้อการทดสอบ** | **วิธีการทดสอบ** | **เกณฑ์ที่ผ่าน (Criteria)** | **ค่าที่วัดได้ / ผลลัพธ์** | **สรุป** |
| **PF-01** | Memory Usage (RAM) | เปิดหน้า Performance Monitor และใช้งานแอป 5 นาที | RAM ต้องไม่เกิน 300MB และไม่เพิ่มขึ้นเรื่อยๆ | เฉลี่ย 150-170 MB | Pass |
| **PF-02** | Frame Rate (FPS) | เลื่อนหน้าจอไปมา หรือกดเปลี่ยนรูปภาพ | UI FPS ต้องเลี้ยงอยู่ที่ 55-60 FPS | 59-60 FPS | Pass |
| **RL-01** | Stress Test (Monkey Test) | กดปุ่มตัวเลือก "ซ้าย/ขวา" สลับกันรัวๆ 20 ครั้ง | แอปต้องไม่เด้งหลุด (No Crash) | แอปทำงานต่อได้ปกติ | Pass |
| **RL-02** | Background Test | ยุบหน้าจอไปเล่นแอปอื่น แล้วกลับเข้ามาใหม่ | แอปต้องแสดงผลหน้าเดิม ไม่รีเซ็ตหรือจอดำ | กลับมาหน้าเดิมได้ | Pass |

## 3.5 ผลการประเมินด้านการบำรุงรักษาและประสิทธิภาพการผลิต

### ผลการทดสอบแอพที่ใช้ Ai พัฒนา

| Ai-Develop | | | |
|---|---|---|---|
| **คุณสมบัติย่อย (Sub-characteristic)** | **เกณฑ์การวัด (Criteria)** | **ผลการดำเนินงานจริง (Actual Result)** | **ผลประเมิน** |
| **1. Modularity (ความเป็นโมดูล)** | มีการแยกส่วนประกอบของโปรแกรมชัดเจน ไม่เขียนโค้ดทั้งหมดลงในไฟล์เดียว | ทำได้ดี : มีการแยกโฟลเดอร์ app, components, constants, context อย่างชัดเจน ทำให้แก้ไขจุดหนึ่งไม่กระทบอีกจุด | ผ่าน |
| **2. Reusability (การนำกลับมาใช้ใหม่)** | มีการเขียนโค้ดที่สามารถนำไปใช้ซ้ำได้ ลดการเขียนโค้ดซ้ำซ้อน | ทำได้ดี : ใช้ AppContext ในการส่งข้อมูลผู้ชนะ และรายการอาหารไปใช้ร่วมกันได้ทุกหน้าจอ โดยไม่ต้องประกาศตัวแปรใหม่ซ้ำซ้อน | ผ่าน |
| **3. Analyzability (ความง่ายในการวิเคราะห์)** | โค้ดอ่านง่าย สามารถหาจุดผิดพลาด (Bug) ได้รวดเร็ว | ทำได้ดี : เมื่อเกิด Error TextImpl ทีมพัฒนาสามารถระบุได้ทันทีว่าเกิดจากไฟล์ food-random.tsx และ food-decision.tsx แต่ยังค่อนข้างสับสนเรื่องชื่อเนื่องจากบางที่มีการตั้งชื่อว่าTournament | ผ่าน |
| **4. Modifiability (ความง่ายในการแก้ไข)** | เมื่อต้องการเปลี่ยน Logic หรือข้อมูลสามารถทำได้โดยไม่ต้องรื้อระบบใหม่ | ทำได้ดีมาก : กรณีเปลี่ยนรูปภาพจาก URL เป็น Local Assets ทีมพัฒนาแก้ไขเพียงจุดเดียวที่ไฟล์ data.ts แล้วระบบอัปเดตอัตโนมัติทั้งแอพ โดยไม่ต้องไปตามแก้ทีละหน้า | ผ่าน |
| **5. Testability (ความสามารถในการทดสอบ)** | โค้ดแยกส่วนกัน ทำให้สามารถทดสอบแยกทีละฟังก์ชันได้ | ทำได้ดี : สามารถทดสอบหน้า Random แยกกับหน้า Tournament ได้อย่างอิสระ เพราะ Logic ไม่ผูกติดกันจนแกะไม่ออก | ผ่าน |

### ผลการทดสอบแอพที่ใช้มนุษย์พัฒนา

| Human-Develop | | | |
|---|---|---|---|
| **คุณสมบัติย่อย (Sub-characteristic)** | **เกณฑ์การวัด (Criteria)** | **ผลการดำเนินงานจริง (Actual Result)** | **ผลประเมิน** |
| **1. Modularity (ความเป็นโมดูล)** | มีการแยกส่วนประกอบของโปรแกรมชัดเจน ไม่เขียนโค้ดทั้งหมดลงในไฟล์เดียว | ทำได้ดี: มีการแยกโฟลเดอร์ app (หน้าจอ), components (ชิ้นส่วน UI), constants (ข้อมูล), context (ตัวแปรระบบ) อย่างชัดเจน ทำให้แก้ไขจุดหนึ่งไม่กระทบอีกจุด | ผ่าน |
| **2. Reusability (การนำกลับมาใช้ใหม่)** | มีการเขียนโค้ดที่สามารถนำไปใช้ซ้ำได้ ลดการเขียนโค้ดซ้ำซ้อน | ทำได้ดี: ใช้ AppContext ในการส่งข้อมูลผู้ชนะ (winner) และรายการอาหาร (selectedFood) ไปใช้ร่วมกันได้ทุกหน้าจอ โดยไม่ต้องประกาศตัวแปรใหม่ซ้ำซ้อน | ผ่าน |
| **3. Analyzability (ความง่ายในการวิเคราะห์)** | โค้ดอ่านง่าย สามารถหาจุดผิดพลาด (Bug) ได้รวดเร็ว | ทำได้ดี: เมื่อเกิด Error TextImpl ทีมพัฒนาสามารถระบุได้ทันทีว่าเกิดจากไฟล์ random-mode.tsx และ decision-mode.tsx เนื่องจากมีการตั้งชื่อไฟล์สื่อความหมายและแยกส่วน Logic/UI ชัดเจน | ผ่าน |
| **4. Modifiability (ความง่ายในการแก้ไข)** | เมื่อต้องการเปลี่ยน Logic หรือข้อมูลสามารถทำได้โดยไม่ต้องรื้อระบบใหม่ | ทำได้ดีมาก: กรณีเปลี่ยนรูปภาพจาก URL เป็น Local Assets ทีมพัฒนาแก้ไขเพียงจุดเดียวที่ไฟล์ data.ts แล้วระบบอัปเดตอัตโนมัติทั้งแอพ โดยไม่ต้องไปตามแก้ทีละหน้า | ผ่าน |
| **5. Testability (ความสามารถในการทดสอบ)** | โค้ดแยกส่วนกัน ทำให้สามารถทดสอบแยกทีละฟังก์ชันได้ | ทำได้ดี: สามารถทดสอบหน้า Random แยกกับหน้า Decision ได้อย่างอิสระ เพราะ Logic ทำงานแยกกัน | ผ่าน |

## 4. ผลด้านคุณภาพ

จากการทดลองข้างต้น สามารถนำกผลจากการพัฒนามาแสดงได้ดังนี้

### 4.1. จำนวนบรรทัด

**หน้า index.ts**

ผลจากการพัฒนาด้วยเอไอ

```
54            </Button>
55          </View>
56        </View>
57      );
58    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
75    chipContainer: { flexDire
76    chip: { marginBottom: 5 }
77    button: { marginTop: 20,
78  });
```

จำนวนรวมการตกแต่ง

ผลจากการพัฒนาด้วยมนุษย์

```
52          </Button>
53          </View>
54        </View>
55        );
56    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
94          marginTop: 20,
95          borderRadius: 10,
96      },
97  }
```

จำนวนรวมการตกแต่ง

**หน้า Mode-select.ts**

ผลจากการพัฒนาด้วยเอไอ

```
24            </Card>
25          </View>
26        </View>
27      );
28    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
40    title: { textAlign: 'cent
41    card: { padding: 10 },
42    cardTitle: { fontWeight:
43  });
```

จำนวนรวมการตกแต่ง

ผลจากการพัฒนาด้วยมนุษย์

```
24            </Card>
25            </View>
26        </View>
27        );
28    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
49  cardTitle: {
50      fontWeight: 'bold',
51      marginBottom: 5 }
52  });
```

จำนวนรวมการตกแต่ง

**หน้า Decision-mode.ts**

ผลจากการพัฒนาด้วยเอไอ

```
86            </Animated.View>
87          </TouchableOpacity>
88        </View>
89      );
90    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
115       zIndex: 20, borderWidth:
116     },
117     vsText: { color: 'white', f
118   });
```

จำนวนรวมการตกแต่ง

ผลจากการพัฒนาด้วยมนุษย์

```
81            </Animated.View>
82          </TouchableOpacity>
83        </View>
84        );
85    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
152     color: 'white',
153     fontWeight: '900',
154     fontSize: 18,
155     fontStyle: 'italic' }
156   });
```

จำนวนรวมการตกแต่ง

**หน้า Random-mode.ts**

ผลจากการพัฒนาด้วยเอไอ

```
69            </TouchableOpacity>
70          </View>
71        </View>
72      );
73    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
103   btnFill: { backgroundCol
104   btnTextOutline: { color:
105   btnTextFill: { color: 'w
106 });
```

จำนวนรวมการตกแต่ง

ผลจากการพัฒนาด้วยมนุษย์

```
74            </TouchableOpacity>
75            </View>
76        </View>
77        );
78    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
150        fontWeight: 'bold',
151        fontSize: 16
152    },
153    });
```

จำนวนรวมการตกแต่ง

**หน้า Results.ts**

ผลจากการพัฒนาด้วยเอไอ

```
86            </Animated.View>
87          </TouchableOpacity>
88        </View>
89      );
90    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
115      zIndex: 20, borderWidth:
116    },
117    vsText: { color: 'white', f
118  });
```

จำนวนรวมการตกแต่ง

ผลจากการพัฒนาด้วยมนุษย์

```
71          </View>
72        </View>
73      );
74    }
```

จำนวนที่ไม่รวมการตกแต่ง

```
134
135    actions: { gap: 10 },
136    btn: { paddingVertical: 5 }
137  });
```

จำนวนรวมการตกแต่ง

## 4.2. จำนวนครั้งที่แก้ไข

เลขจำนวนครั้งสำหรับการแก้ไขโค้ดที่ได้พัฒนาไป โดยบันทึกทั้ง อาการ ความรุนแรง สาเหตุ ครั้งที่แก้ โดยที่ความรุนแรงที่ได้ประเมินนั้นมีเกณฑ์ดังนี้

- High คือ ปัญหาที่รุนแรงส่งผลให้แอพใช้งานไม่ได้
- Medium คือ ปัญหาที่ส่งผลต่อระบบ แต่แอพยังคงใช้งานได้
- Low คือ ปัญหาแจ้งเตือนเล็กน้อย แอพยังคงสามารถทำงานได้ถูกต้อง

ผลการทดสอบแอพที่ใช้ Ai พัฒนา

| Defect ID | อาการที่พบ (Bug Description) | ความรุนแรง | สาเหตุ | สถานะ | รอบการแก้ (Rework) |
|---|---|---|---|---|---|
| BUG-001 | App เด้งพร้อม Error [Worklets] Mismatch | High (Critical) | Library reanimated เวอร์ชันใหม่เกินกว่าที่ Expo Go รองรับ | Fixed | 2 รอบ |
| BUG-002 | หน้าสุ่มอาหาร Error Invariant Violation: TextImpl | High (Critical) | Library react-native-paper ไม่รองรับการทำ Animation ร่วมกับ reanimated | Fixed | 3 รอบ |
| BUG-003 | โหมด Tournament จบการทำงานทันทีทั้งที่ยังแข่งไม่จบ | High (Critical) | Logic การคำนวณคู่แข่งขันผิดพลาดเมื่อจำนวนอาหารเป็นเลขคี่ (Odd numbers) | Fixed | 1 รอบ |
| BUG-004 | Error Missing default export / หน้าจอขึ้น Unmatched Route | Medium | Cache ของระบบ (Metro Bundler) จำค่าไฟล์เก่าที่ Error ไว้ ไม่ยอมอัปเดตโค้ดใหม่ | FIxed | 1 รอบ |
| BUG-005 | รูปภาพอาหารไม่แสดง | Medium | การใช้ URL รูปภาพจากอินเทอร์เน็ตโหลดไม่ทัน/ติด Permission | Fixed | 1 รอบ |
| BUG-006 | ตัวหนังสือในหน้า Result ทับกัน และมี Warning เรื่อง Shadow | Low | Component Card.Title ไม่ยืดหยุ่น และ overflow: hidden ไปตัดเงาออก | Fixed | 1 รอบ |

ผลการทดสอบแอพที่ใช้มนุษย์พัฒนา

| Human-Develop | | | | | |
|---|---|---|---|---|---|
| Defect ID | อาการที่พบ (Bug Description) | ความรุนแรง | สาเหตุ | สถานะ | รอบการแก้ (Rework) |
| BUG-001 | App เด้ง (Crash) พร้อม Error [Worklets] Mismatch | High (Critical) | Logic ในหน้า Decision ผิดพลาดเมื่อเหลือตัวเลือก 1 ตัว | Fixed | 2 รอบ |
| BUG-002 | Error: Rendered fewer hooks than expected | High (Critical) | ใส่ return ก่อนเรียก useEffect ในหน้า Result | Fixed | 1 รอบ |
| BUG-003 | ปุ่มกดแล้วเลือกผิดฝั่ง | Medium | ส่ง Parameter ใน onPress ผิด (ส่ง Left ให้ปุ่มล่าง) | Fixed | 1 รอบ |
| IMPROVE-1 | เพิ่ม Animation | Low | ผู้ใช้ต้องการให้ดูลื่นไหลขึ้น | Done | - |

# บทที่ 5

## สรุป อภิปรายผลการศึกษา และ บทคัดย่อ

### 1. สรุปผลการศึกษา

**1.1 ด้านประสิทธิภาพในการผลิต** การใช้เอไอ มีความโดดเด่นในเรื่องของความรวดเร็วในการพัฒนาอย่างมาก

เวลาที่ใช้ทั้งหมด: เอไอใช้เวลาทั้งหมด 88 นาที ในขณะที่มนุษย์ใช้เวลา 295 นาที

ระยะเวลาการขึ้นโครงร่างช่วงที่ 1 : เอไอใช้เวลาเพียง 48 นาที ซึ่งเร็วกว่ามนุษย์ที่ใช้เวลาถึง 245 นาที สาเหตุหลักมาจาก AI สามารถเจนเนอเรตโค้ดจำนวนมากได้ในทันทีโดยไม่ต้องพิมพ์เองทีละบรรทัด

ระยะเวลาการแก้ไขช่วงที่ 2 : เอไอใช้เวลาในการแก้ไข 40 นาที ซึ่งใกล้เคียงกับมนุษย์ที่ใช้เวลา 50 นาที ชี้ให้เห็นว่าแม้ เอไอจะเขียนโค้ดเร็ว แต่ใช้เวลาในการแก้บั๊ก ในสัดส่วนที่ค่อนข้างสูงเมื่อเทียบกับเวลาที่ใช้สร้าง

**1.2 ด้านคุณภาพของซอฟต์แวร์** จากการประเมินตามมาตรฐาน ISO/IEC 25010 พบว่าแอพพลิเคชั่นทั้งสองแบบมีคุณภาพที่ใกล้เคียงกันเมื่อเสร็จสมบูรณ์

**ความถูกต้อง** : แอพที่พัฒนาด้วย AI พบข้อผิดพลาด จำนวน 6 รายการ ซึ่งมีความรุนแรงระดับความเสี่ยงสูงถึง 3 รายการ ส่วนใหญ่เกิดจากปัญหาเวอร์ชั่นไม่สอดคล้องกัน ของไลบรารีและตรรกะการคำนวณเลขคี่ในโหมด Tournament ในขณะที่มนุษย์พบข้อผิดพลาด 3 รายการ

**โครงสร้างโค้ด** : โค้ดที่สร้างโดย AI มีความเป็นระเบียบ มีการแยกไฟล์และคอมโพเน้น อย่างชัดเจน ใกล้เคียงกับมาตรฐานที่มนุษย์ทำ ทำให้ง่ายต่อการบำรุงรักษา

### 2. อภิปรายผลการศึกษา

#### 2.1 ปริมาณโค้ดกับการจัดการ

เอไอสามารถเขียนโค้ดได้รวดเร็วและดีตามพรอมต์ที่ได้รับ แต่ว่ายังคงมีจุดอ่อนคือโค้ดที่ไม่จำเป็นหรือไม่ได้ใช้บางส่วน ความรู้ที่เป็นปัจจุบัน และบริบทของเครื่องมือเช่น ในการทดลองเอไอมีการแนะนำให้ใช้ไลบรารี่ที่มีปัญหา ที่ไม่สามารถใช้งานได้กับเวอร์ชั่นปัจจุบันของExpo Go ซึ่งสะท้อนให้เห็นได้ว่า เอไอนั้นไม่ได้ทำการทดสอบด้วยโค้ดจริง แต่เป็นการคาดเดาจากแพทเทิร์นข้อมูลที่มีอยู่ ทำให้มนุษย์ต้องเข้ามาจัดการปัญหาทางเทคนิค

#### 2.2 บทบาทของ Human-in-the-Loop

ผลการทดลองยืนยันแนวคิด Human-in-the-Loop ว่ามีความสำคัญอย่างยิ่ง ถึงแม้ว่าในช่วงการพัฒนาแอพพลิเคชั่น เอไอทำได้ดีกว่ามนุษย์ถึง 5 เท่า แต่ว่าในช่วงที่แก้ไขและบำรุงรักษานั้น มนุษย์และเอไอยังคงใช้

เวลาเท่าๆกัน สำหรับการไล่หาจุดผิดพลาดและแก้ไข หากการทำงานด้วยเอไอไม่มีมนุษย์ที่มีความรู้เข้ามาตรวจสอบ แอพพลิเคชั่นที่พัฒนาด้วยเอไอ อาจจะไม่สามารถใช้งานได้จริงเนื่องจากข้อผิดพลาดที่ร้ายแรง

**2.3 ประสิทธิภาพต่อความซับซ้อน**

สำหรับฟังชั่นพื้นฐานเช่น หน้าการสุ่มธรรมดานั้น เอไอสามารถทำงานได้ถูกต้องแม่นยำ แต่เมื่อเจอหน้าตัดสินใจ ที่เป็นการเลือกจับคู่หาคนแพ้ชนะ ที่มีตรรกะที่ซับซ้อนมากขึ้นนั้น เอไอจะเริ่มแสดงข้อผิดพลาดทางตรรกะให้เห็น แสดงให้เห็นว่าเอไอนั้น เหมาะกับการทำงานที่เป็นงานทำซ้ำๆที่มีรูปแบบชัดเจน มากกว่างานที่ต้องใช้ตรรกะซับซ้อนเฉพาะทาง

**3. ข้อเสนอแนะ**

**3.1 ข้อเสนอแนะในการนำไปใช้งาน**

**ใช้เอไอเป็นผู้ช่วยงาน ไม่ใช่ผู้ทดแทน** ควรใช้เอไอสำหรับการทำงานในเบื้องต้นเช่น การทำฟังชั่นเบื้องต้น การตกแต่งหน้าเว็บเบื้องต้น หรือการแนะไอเดียในช่วงเริ่ม เพื่อลดเวลาในการทำงานบางส่วนสำหรับงานที่เป็นงานเบื้องต้นและเป็นงานที่มีการทำซ้ำเยอะ ไม่เฉพาะทาง สำคัญเลยคือไม่ควรคาดหวัง ให้เอไอทำงานแทนจนจบกระบวนการ โดยไม่มีการตรวจสอบ

**ระบุเวอร์ชั่นให้ชัดเจนในพรอมต์** เพื่อลดปัญหาข้อผิดพลาดด้านการทำงานร่วมกันของเครื่องมือ ผู้ใช้ควรระบุเวอร์ชั่นของชุดเครื่องมือที่ใช้พัฒนาให้ชัดเจน ลงบนคำสั่งพรอมต์ตั้งแต่แรก

**นักพัฒนาต้องมีความรู้พื้นฐาน** การใช้เอไอที่ดีและมีประสิทธิภาพที่สุดนั้น ผู้ใช้จำเป็นต้องมีความรู้ในการอ่านโค้ดและแก้ปัญหา เพื่อแก้ไขข้อผิดพลาดที่เอไอสร้างขึ้นได้รวดเร็ว

**3.2 ข้อเสนอแนะสำหรับการศึกษาในอนาคต**

**ควรเพิ่มความซับซ้อนของโจทย์** เพื่อทำการทดสอบขีดจำกัดของเอไอ ในการที่มีขนาดใหญ่และมีความซับซ้อนมากขึ้น

**ควรทดสอบกับเอไอโมเดลอื่นๆ** เพื่อเป็นการเปรียบเทียบได้ว่าเครื่องมือตัวไหน สามารถใช้งานได้มีประสิทธิภาพมากที่สุด

**บทคัดย่อ**

โครงงานนี้มีวัตถุประสงค์เพื่อศึกษาและเปรียบเทียบคุณภาพและประสิทธิภาพการผลิต ในการพัฒนาซอฟต์แวร์ระหว่างการใช้ปัญญาประดิษฐ์และการพัฒนาโดยมนุษย์ โดยทำการทดลองพัฒนาโมบายแอพพลิเคชั่น "ระบบตัดสินใจเลือกอาหาร"ด้วย ครอสแพลทฟอร์ม เฟรมเวิร์ค โดยกำหนดความต้องการเดียวกัน

ผลการศึกษาพบว่า ในด้าน ประสิทธิภาพการผลิต การใช้เอไอ ช่วยลดระยะเวลาในการพัฒนาโดยรวมได้อย่างมีนัยสำคัญ โดยใช้เวลาเพียง 88 นาที เมื่อเทียบกับมนุษย์ที่ใช้เวลา 295 นาที คิดเป็นความรวดเร็วที่เพิ่มขึ้นกว่า 3 เท่า อย่างไรก็ตาม ในด้าน คุณภาพและความถูกต้อง แอพพลิเคชั่นที่พัฒนาด้วยเอไอ พบข้อผิดพลาด ในระหว่างการพัฒนาจำนวน 6 รายการ ซึ่งมากกว่ามนุษย์ที่พบเพียง 3 รายการ โดยเฉพาะปัญหาเรื่องความเข้ากันได้ของไลบรารีและตรรกะที่ซับซ้อน

สรุปได้ว่า เอไอมีประสิทธิภาพสูงมากในการขึ้นโครงร่างโปรแกรมและการเขียนโค้ดพื้นฐาน แต่ยังขาดความแม่นยำในการจัดการกับเวอร์ชั่น ของเครื่องมือและการแก้ปัญหาเฉพาะหน้า จึงสรุปได้ว่ารูปแบบการทำงานที่มีประสิทธิภาพที่สุดคือ Human-in-the-Loop หรือการใช้เอไอช่วยสร้างและมนุษย์เป็นผู้ตรวจสอบและแก้ไขเพื่อให้ได้งานที่รวดเร็วและมีคุณภาพควบคู่กัน

**อ้างอิง**

Surapapha, P., & Paireekreng, W. (2024). A comparative study of efficiency and quality of software project deliverables in Thai government agencies: ISO/IEC 29110 vs. 3-phase quality inspection with ISO/IEC 29110 [In Thai]. Journal of Business Administration, 14(2), 65–82.

Sauvola, J., Tarkoma, S., Klemettinen, M., Riekki, J., & Doermann, D. S. (2024). Future of software development with generative AI. Automated Software Engineering, 31(1), Article 26. https://doi.org/10.1007/s10515-024-00426-z

Farias, J. P., et al. (2021). Tool for Measuring Productivity in Software Development Teams. Information, 12(10), 396. https://doi.org/10.3390/info12100396